
ALGORITMOS E ESTRUTURA DE DADOS

Teoria e Implementações

J. P. S. DE SOUSA

Sumário

I	Estruturas de Dados Elementares	1
1	Estruturas Sequenciais	2
1.1	Vetores	3
1.1.1	Vetores Não Ordenados	3
1.1.2	Vetores Ordenados	4

PARTE I

ESTRUTURAS DE DADOS ELEMENTARES

Estruturas Sequenciais

Na matemática, conjuntos são coleções imutáveis de objetos distintos. Na ciência da computação, conferimos a nomenclatura de **conjuntos** a qualquer coleção finita abstrata de objetos. A imutabilidade, nesse caso, não é um requisito; conjuntos computacionais podem crescer ou diminuir de tamanho ao longo das operações de um algoritmo que o manipula. Tais conjuntos que podem ter seu tamanho alterado chamamos de **dinâmicos**. Os conjuntos dinâmicos que também permitem o teste de pertinência de um membro dado são chamados de **dicionários**.

Os elementos de um conjunto de dados são, geralmente, um registro, cujos campos podem ser de tipos elementares, ponteiros, ou compostos de outros registros. Os elementos num conjunto não necessariamente precisam ser todos distintos entre si, a exemplo da coleção de idades de um grupo de pessoas; nada impede que haja duas com a mesma idade. Em alguns casos, os membros de um conjunto possuem uma **chave** que os identifica, e, caso as chaves sejam todas distintas, o conjunto pode ser considerado como uma coleção de chaves.

As operações de um conjunto podem ser de dois tipos: consulta ou modificação. A primeira apenas resgata um valor, membro ou estatística do conjunto. A segunda altera o estado do conjunto, como sua ordenação, inserindo, removendo ou modificando membros, entre outras. Em geral, quase todo conjunto aceita as seguintes operações:

Insere(S, x) Operação que toma um conjunto S e insere o membro apontado por x . Presume-se que tanto S quanto x já foram inicializados. Dependendo da aplicação, a operação pode falhar caso um membro x com a mesma chave já exista em S .

Remove(S, x) Remove do conjunto S o membro apontado por x .

Busca(S, k) Retorna um ponteiro para um membro x em S cuja chave seja igual a k , ou **NIL** caso nenhum elemento contenha tal chave.

Outras operações básicas que acompanham conjuntos dinâmicos são

Máximo(S) Retorna o membro x de S cujo valor da chave é máximo no conjunto

Mínimo(S) Retorna o membro x de S cujo valor da chave é mínimo no conjunto

Predecessor(S, x) Num conjunto ordenado, retorna o membro $w \in S$ que precede x , ou **NIL** quando x é o elemento mínimo.

Sucessor(S, x) Num conjunto ordenado, retorna o membro $y \in S$ que sucede x , ou **NIL** quando x é o elemento máximo.

1.1 Vetores

Um vetor é um conjunto de objetos de mesmo tipo¹ que são armazenados contiguamente na memória. Seus elementos podem ser acessados diretamente pelos seus índices. Se o nome do vetor é S , então acessamos seu i -ésimo membro como $S[i]$. O nome do vetor é um ponteiro para o seu primeiro elemento, e, ao somá-lo com um valor inteiro não negativo i , obtemos um ponteiro para o elemento de índice i do vetor, portanto $S[i] = *(S + i)$.

Na sua formulação mais simples, vetores possuem dois atributos principais: tamanho – o número máximo de elementos que ele suporta – e o número efetivo de membros. Chamamos de estáticos os vetores que possuem tamanho previamente determinado e fixo, enquanto são denominados dinâmicos aqueles cujo tamanho varia conforme a necessidade da aplicação. Além disso, os vetores podem ser ordenados ou não.

1.1.1 Vetores Não Ordenados

A busca em vetores não ordenados é realizada percorrendo o vetor sequencialmente até achar o elemento com a chave desejada. No pior caso, o elemento ou está na última posição ou não existe no vetor. Com isso, a complexidade de uma busca sequencial no vetor com n elementos é $O(n)$.

Função Busca-Vetor(S, k)	
1	$i \leftarrow 0$;
2	enquanto $i < S.\text{num_elem}$ e $S[i].\text{chave} \neq k$ faça
3	$i \leftarrow i + 1$;
4	fim
5	se $i = S.\text{num_elem}$ então
6	retorna NIL ;
7	fim
8	retorna $\&S[i]$;

A inserção em vetores não ordenados pode ser feita apenas inserindo após o último elemento. Já a operação de remoção pode ser realizada simplesmente copiando o último elemento para a posição do elemento que está sendo removido. Ambas são realizadas

¹Isso porque dois dados do mesmo tipo possuem o mesmo número de bytes na memória.

em tempo $O(1)$, mas como pode ser necessário buscar o elemento no vetor antes de o remover, a remoção dele pode custar $O(n)$.

Função Insere-Vetor(S, x)
<pre>1 se $S.len = S.num_elem$ então 2 erro “overflow”; 3 fim 4 copia($S + S.num_elem, x$); 5 $S.num_elem \leftarrow S.num_elem + 1$;</pre>
Função Remove-Vetor(S, x)
<pre>// x deve apontar para um elemento do vetor: $S \leq x < S + S.num_elem$ 1 se $S.num_elem = 0$ então 2 erro “underflow”; 3 fim 4 $S.num_elem \leftarrow S.num_elem - 1$; 5 $ultimo \leftarrow S + S.num_elem$; 6 copia($x, ultimo$);</pre>

1.1.2 Vetores Ordenados

Em vetores ordenados, a busca pode ser feita em tempo sublinear $O(\log n)$ se usarmos o algoritmo de busca binária, que será apresentado num capítulo mais adiante. Uma abordagem possível para a inserção é inserir o novo membro ao final do vetor, e o movimentar na direção decrescente enquanto sua chave for menor que a do membro na posição anterior. Para remoção, o procedimento consiste em deslocar todos os membros à frente daquele que foi removido para uma posição atrás. Ambas as operações passam a ter, pois, complexidade $O(n)$, já que necessitam mover todos os membros do vetor no pior caso.

Função Insere-Vetor-Ordenado(S, x)
<pre>1 se $S.len = S.num_elem$ então 2 erro “overflow”; 3 fim 4 copia($S + S.num_elem, x$); 5 $i \leftarrow S.num_elem$; 6 enquanto $i \geq 1$ e $S[i].chave < S[i - 1].chave$ faça 7 $temp \leftarrow S[i - 1]$; 8 $S[i - 1] \leftarrow S[i]$; 9 $S[i] \leftarrow temp$; 10 $i \leftarrow i - 1$; 11 fim 12 $S.num_elem \leftarrow S.num_elem + 1$;</pre>

Função Remove-Vetor-Ordenado(S, x)

```

1 se  $S.\text{num\_elem} = 0$  então
2   | erro “underflow”;
3 fim
4 para  $i \leftarrow x - S$  até  $S.\text{num\_elem} - 2$  faça
5   |  $S[i] \leftarrow S[i + 1]$ ;
6 fim
7  $S.\text{num\_elem} \leftarrow S.\text{num\_elem} - 1$ ;

```

Operações para obter o máximo e o mínimo, basta retornar o ponteiro para o último e o primeiro elemento no vetor respectivamente. Para obter o predecessor de um elemento, devemos apenas subtrair 1 ao ponteiro dado como argumento caso esse não seja o último. Para o sucessor, devemos somar 1 a esse ponteiro, caso ele não aponte para o último.

```

1 função Maximo( $S$ )
2   | retorna  $S + S.\text{num\_elem} - 1$ ;
3 fim
4 função Minimo( $S, x$ )
5   | retorna  $S$ ;
6 fim
7 função Predecessor( $S, x$ )
8   | se  $x = \text{Minimo}(S)$  então
9     | retorna NIL;
10  fim
11  retorna  $x - 1$ ;
12 fim
13 função Sucessor( $S, x$ )
14   | se  $x = \text{Maximo}(S)$  então
15     | retorna NIL;
16  fim
17  retorna  $x + 1$ ;
18 fim

```
